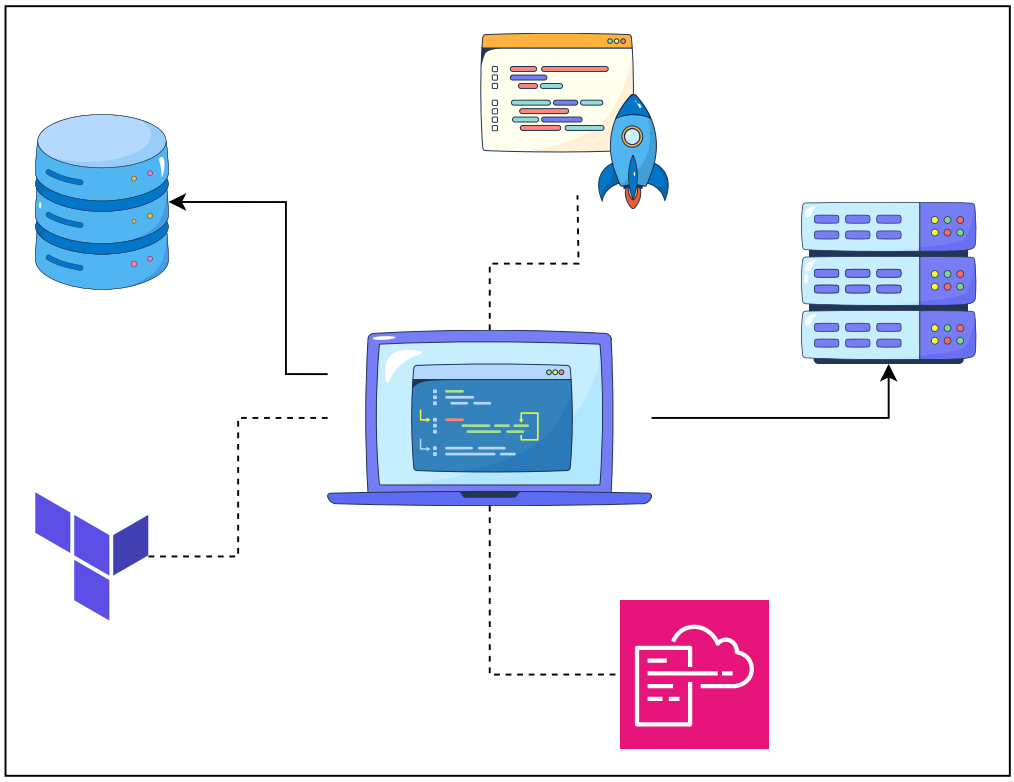
Infrastructure as Code with CloudFormation

Explore how AWS CloudFormation allows you to define and automate cloud infrastructure through code, ensuring repeatability, version control, and safe updates. Understand template structure, stack sets, and best practices to efficiently manage AWS resources and integrate deployments into CI/CD pipelines.

Infrastructure as code is a method of provisioning and managing cloud resources by writing templates instead of clicking through the AWS Console. This approach ensures consistency, repeatability, and version control, just like source code.

For instance, a development team working on a microservices-based application can use CloudFormation templates to deploy identical environments across dev, staging, and production while ensuring that each environment reflects the same architecture, resource configurations, and security settings without manual drift or oversight.

Ask



For developers, the key benefits include:

- Automated resource provisioning with no manual steps.
- Version-controlled infrastructure using Git or other source control systems.
- Reusable, modular templates that can be shared across teams.
- Reduced human error, especially in a complex environment.

How CloudFormation fits into the AWS developer ecosystem

AWS CloudFormation is a foundational IaC service that integrates deeply with the AWS ecosystem. It allows us to define infrastructure in YAML or JSON and provision it in a controlled and predictable manner. This makes it ideal for developers seeking automation, versioning, and repeatability across development, staging, and production environments.

Ask

As a developer, CloudFormation enables us to:

- Define infrastructure in source-controlled templates. This enables easy rollback, auditing, and collaboration through standard version control systems like Git.
- Automate deployments via the AWS CLI, SDKs, or AWS Console. It reduces deployment time and ensures resources are provisioned consistently.
- Integrate infrastructure deployments into CI/CD pipelines. This allows teams to deploy infrastructure changes alongside application code for end-to-end delivery automation.
- Use parameters and conditions to create flexible and reusable templates. This helps customize stacks for different environments or workloads using the same base template.

CloudFormation is fully supported across AWS regions and can manage nearly every AWS resource.

Sharing infrastructure as code across multiple AWS accounts

To further improve the reusability, we can share infrastructure as code across multiple AWS accounts in an Organizational Unit. This can be achieved using three components:

- **CloudFormation module:** It is a bundle of related AWS services.
- **CloudFormation template:** The main blueprint of the complete set of resources.

Ask

- **CloudFormation stack set:** A Stack Set is a management tool that lets us take a single CloudFormation template and deploy it across multiple AWS accounts and regions simultaneously.

Using these components, we can set up the infrastructure to allow sharing of infrastructure as code among multiple accounts, as shown in the architecture diagram below:

Architecture diagram for sharing infrastructure as code across multiple accounts

Here, the first CloudFormation module is created and pushed into an S3 bucket. Then, we create a CloudFormation template to reference this module. This template is used to create a stack set, which is deployed across multiple accounts and regions.

Declarative templates: YAML and JSON

CloudFormation templates are written in YAML or JSON, both of which allow a declarative approach to defining infrastructure. Rather than writing instructi step-by-step (imperative), we declare the desired state of our resources. For example, a DevOps team can define a scalable web application architecture,

Ask

including load balancers, EC2 instances, and auto-scaling groups in a single YAML file, making it easy to replicate across multiple regions or projects.

Writing basic templates

CloudFormation templates follow a structured format, allowing us to define what AWS resources we want, without having to describe how to create them step-by-step. At a minimum, a basic template must contain a `Resources` section, which is the heart of the template where all infrastructure components are defined. We may also include optional sections like `Description`, `Parameters`, and `Outputs` to enhance readability, flexibility, and integration.

A basic template has the structure as shown below:

index.yaml YAML

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: Basic CloudFormation Template
Resources:
  MyBucket:
    Type: AWS::S3::Bucket
```

Basic CloudFormation template

YAML vs. JSON

Ask

Cloudformation supports both YAML and JSON templates but when to use what?

YAML is more human-readable and supports comments. Its clean syntax makes it easier for teams to review and maintain templates collaboratively, especially when onboarding new developers.

JSON, on the other hand, is more verbose and error-prone but widely supported in automation tools. It is strict about formatting, which can help catch certain syntax issues early, but may also lead to frustration with large or nested structures.

Defining AWS infrastructure with resources

The `Resources` section is the core of every CloudFormation template. Here, we define the AWS components we want to create. Each resource is defined with a logical name, a type (like `AWS::S3::Bucket` or `AWS::EC2::Instance`), and a set of properties that configure its behavior.

Template core elements

Let's examine some of the core elements of a CloudFormation template:

- **AWSTemplateFormatVersion:** Template version identifier. This is typically set to `'2010-09-09'`, the only valid value as of now, which helps ensure forward compatibility.
- **Description:** Optional text description. It helps document the purpose of the stack or template for team members and future maintainers.
- **Resources:** Mandatory section defining AWS resources. This is where we define each AWS service we want CloudFormation to create and manage as part of our infrastructure stack.

The code snippet below shows the YAML template to deploy a `t2.micro` EC2 instance:

Ask

index.yaml YAML

```yaml
Resources:
  MyInstance:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: t2.micro
      ImageId: ami-0abcdef1234567890
```

YAML template to deploy a t2.micro

Logical names like `MyInstance` are used to refer to resources within the stack.

- **Parameters:** The `Parameters` section allows us to inject values at deployment time, making templates customizable:

index.yaml YAML

```yaml
Parameters:
  InstanceType:
    Type: String
    Default: t2.micro
```

Ask

Parameters in CloudFormation template

- **Mappings:** They define static, region-specific or environment-specific values:

index.yaml YAML

```yaml
Mappings:
  RegionMap:
    us-east-1:
      AMI: ami-abc123
    us-west-1:
      AMI: ami-def456
```

Mappings in CloudFormation template

Both Parameters and mappings enable developers to make the templated more reusable.

- **Outputs:** We can declare outputs to expose values like resource IDs, ARNs, or IPs after stack creation:

index.yaml YAML

```yaml
Outputs:
  InstanceId:
    Value: !Ref MyInstance
    Export:
      Name: MyInstanceId
```

Ask

Outputs in CloudFormation template

- **Conditions:** Use the `Conditions` section to control whether resources are created:

```
Conditions:
  CreateProdResources: !Equals [ !Ref EnvType, "prod" ]
```

Conditions in CloudFormation template

- **Metadata:** Metadata adds extra info for tools or documentation.

index.yaml YAML

```yaml
Metadata:
  AWS::CloudFormation::Interface:
    ParameterGroups:
      - Label:
          default: "Environment Configuration"
        Parameters:
          - EnvType
```

Metadata in CloudFormation template

We can create or update a CloudFormation stack named `my-stack` using the CLI command given below:

Ask

index.yaml YAML

```yaml
aws cloudformation deploy \
  --template-file mytemplate.yaml \
  --stack-name my-stack \
  --parameter-overrides InstanceType=t2.micro \
  --capabilities CAPABILITY_NAMED_IAM
```

AWS CLI command to deploy the stack

The given command is a streamlined way to deploy or update a CloudFormation stack using the AWS CLI. It leverages the `aws cloudformation deploy` command to apply a template file named `mytemplate.yaml`, which defines the infrastructure as code. The stack is named `my-stack`, and the deployment includes an override for the `InstanceType` parameter, setting it to `t2.micro`. This allows flexible reuse of templates across environments by customizing values at deploy time. The `--capabilities CAPABILITY_NAMED_IAM` flag is crucial—it grants CloudFormation permission to create or modify IAM resources that have explicitly defined names. Without this flag, any stack containing such resources would fail to deploy. This approach is well-suited for automating infrastructure provisioning in CI/CD pipelines and aligns directly with the Deployment domain of the DVA-C02 exam.

Change sets: Safer updates in production

When we update a stack, instead of applying infrastructure updates directly, CloudFormation's change sets let us preview their exact impact first. This "look before you leap" approach is a crucial safety check for production environments. The process is simple:

Ask

1. **Create a change set:** We start by generating a change set with our updated template. For example, if we're changing an EC2 instance type for performance tuning, we would first create a change set to see the proposed modifications.
2. **Review the execution plan:** The change set details every resource that will be added, modified, or deleted. This review is our chance to prevent unintended consequences, like the accidental removal of a critical database or load balancer.
3. **Execute:** Once we have confirmed the plan is safe, we execute the change set. The update then proceeds exactly as previewed, ensuring a controlled and predictable rollout of our new application version or infrastructure tweak.

This provides a safer path for production deployments.

Cross stacks

In AWS CloudFormation, a cross-stack reference allows one CloudFormation stack to use resources like outputs from another stack. This helps us break a large infrastructure into modular, reusable stacks, making the templates easier to manage and maintain.

For example, instead of creating a single stack that includes all resources, we may divide our infrastructure into two stacks: one for networking, including VPC, subnets, and security groups, and the other for deploying the application infrastructure.

Now, if we want to refer to the security group for the EC2 instance in the application stack, we add the following configuration in the network stack.

index.yaml YAM Ask

```
Outputs:
  AppSGOutput:
    Description: Security Group ID to be used by other stacks
```

```
      Value: !Ref AppSecurityGroup
      Export:
        Name: AppSecurityGroupID
```

Sample CloudFormation template of network stack

Then, in the application stack, we import the security group using `Fn::ImportValue` like this:

index.yaml YAML

```yaml
Resources:
  AppInstance:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-0abcdef1234567890
      InstanceType: t3.micro
      SecurityGroupIds:
        - !ImportValue AppSecurityGroupID
      SubnetId: subnet-0123456789abcdef0
```

Sample CloudFormation template for application stack

This setup allows the application stack to reuse resources defined in the network stack without duplicating code.

Ask

Visual tools

In addition to the CLI, developers often leverage visual tools to simplify stack creation, editing, and debugging during development and deployment.

- **AWS Console:** Step-by-step UI for deploying stacks. It is useful for beginners or quick prototyping, offering guided stack creation with form-based inputs and error validation.
- **VS Code extensions:** Plugins like AWS Toolkit for syntax highlighting, deployment, and validation streamline developer workflows by enabling template editing, stack management, and direct AWS account integration within the IDE.

Troubleshooting

Here are some of the tips for troubleshooting CloudFormation templates:

- Use `aws cloudformation describe-stack-events` for error tracing. This command provides a detailed timeline of stack operations, helping us

Ask